高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



复杂FA算子Agent构建

张峰

盘古大模型 技术专家



盘古大模型 + Agent：CANNBench 排行榜实践

CANNBOT_direct_92（盘古 92B） · 950DT / v1.1 官网筛选结果



CANNBench：FA 相关任务覆盖了哪些能力？

从注意力主体到前处理：5 个 L4 相关任务

分类	官方任务名称	计算范围	工程关注点
注意力主体	MHA	已分头 Q/K/V 的缩放点积注意力	多头布局、因果掩码、序列边界
注意力主体	GQA	多个 Query 头共享 KV 头	分组映射、KV 复用与访存
注意力主体	MLA	内容 / 位置分量的注意力计算 不含 KV 解压缩	维度组合、RoPE 分量与布局
直接 FA 相关	SparseFlashAttention	按稀疏索引选取 KV 子集 执行缩放点积注意力	索引 Gather、离散访存、掩码
注意力前处理	MlaProlog	投影 + RMSNorm + RoPE 不包含后续 Attention 主体	多步融合、计算与数据依赖

FA 是一种实现方法；MHA / GQA / MLA 是注意力计算语义，评测任务名称不限定必须采用 FA 实现。

对应 Agent 能力：理解任务契约 → 选择分块 / 融合方案 → 验证精度与边界 → 定位同步和访存问题

背景与痛点 → 核心思路 · 端到端总体架构

三大痛点

①开发门槛高
AscendC 编程模型复杂：Tiling 切分、多核协同、核内流水、CV并行、同步机制、UB\L1\L0 内存分配和复用

②开发周期长
设计→开发→联调→验收环节多、人工反复迭代；tiling/host-kernel/多核/硬件差异牵一发动全身

③问题定位难
精度/同步/内存问题依赖个人经验；经验散落 wiki、笔记、PR ， 老问题反复定位、新人无处可查

三条主线

① AI辅助研发
设计·编码·测试·Review·定位·调优，端到端建设 Agent/Skill 体系

② 知识驱动AI
从「静态文档」到「固化的人类经验」， 建设案例 & 知识驱动型 SKILL

③ 质量门禁体系
正向建设 + 反向门禁（阶段硬门禁 / 编译判据 / 审查打回）

算子开发Agent的流水线

需求/Golden → 概要设计 → 详细设计 → 测试设计与用例生成 → 代码开发 → 实现审查 → 精度调试 → 交付验收 → 知识沉淀

质量保障：用 Harness 方法论来建设 Agent 的质量保障体系

- ✓ **Harness 是把「信任」变成「检查」的那层壳**
- ✓ **工程落点：每一件都对应具体文件**

分工 谁写 / 谁读，边界写死	▪ 1 个主调度Agent + N 个SubAgent ▪ 主调度不写代码、不审查、不定位失败；「谁跑测试、谁改代码」均拆分明确
契约 交付件有 Schema 约定	▪ artifacts/ 目录契约：15+ 模板化交付件（op_spec / case_plan / handoff…），瞬态产物进 .debug/ ▪ CONTEXT.md 唯一 owner = planner；implementation_handoff 追加式交接
门禁 可检查的成果物才放行	▪ 编译门禁：Cube 编译通过，才准碰 Vector；产物门禁：只查「文件在不在」 ▪ 字段级路由：review_gate 只读「结论」、debug_state 只读「状态」—— 一个字决定走哪条边
证据 决策留痕，一切可核查	▪ debug_packet.jsonl：每跑一次验证追加一行 —— 上一条没写入，禁止继续分析代码 ▪ confirmed_facts 必须具体，不许写「看起来有问题」；baseline / last_good 多档 checkpoint
学习 经验沉淀成资产	▪ knowledge-curator 强制调度：playbooks / index / patterns 三层知识库，index 限行防膨胀 ▪ 先查库才开工，采用 / 拒绝都记 knowledge_usage.md；不生成 trace.md 不算竣工

环节①：概要设计—选对路线

概要设计六步流程

① 场景解析与流图构建
需求场景 + Golden → 识别算子类型，生成计算流图

② API映射与类型识别
Golden → Ascend API 映射 + 算子类型识别

③ 相似案例复用
检索相似真实案例，生成初稿概要方案

④ 优化点识别
基于优化知识库，自动识别关键优化点

⑤ 方案设计与筛选
设计候选方案，按收益 / 可行性筛选

⑥ 方案汇总与校验
聚合优化点输出最终方案，规则库自动校验修正

核心挑战

- **流水设计难**
  算子内计算 / 访存（CV）互相掩盖困难，Tiling 分块大小难精准把控
- **场景组合杂**
  应用场景多、算子特性多，交叉影响点复杂，难以穷举分析
- **硬件耦合深**
  与芯片架构强相关，依赖芯片知识，知识库规模大、维护成本高

关键点

- **知识库驱动**
  沉淀芯片知识与历史案例，降低专家经验依赖
- **案例冷启动**
  相似案例快速复用，提升方案合理性与生成效率
- **优化闭环**
  识别—设计—筛选自动化，减少人工试错
- **结构化输出**
  标准计算流图与设计文档，便于评审与落地

落地效果

维度	输入依据	输出成果	验证
需求分析	需求 / Golden	计算流图	**可追溯**
需求设计	案例 / 知识库	候选方案	**可评审**

总结

通过知识库驱动的「案例参考—优化识别—自动校验」闭环，
将 attention 算子概要设计从依赖专家经验的手工过程，
升级为可复用、可校验、可落地的自动化生成，显著提升设计效率与方案质量。

环节②：详细设计—定好契约

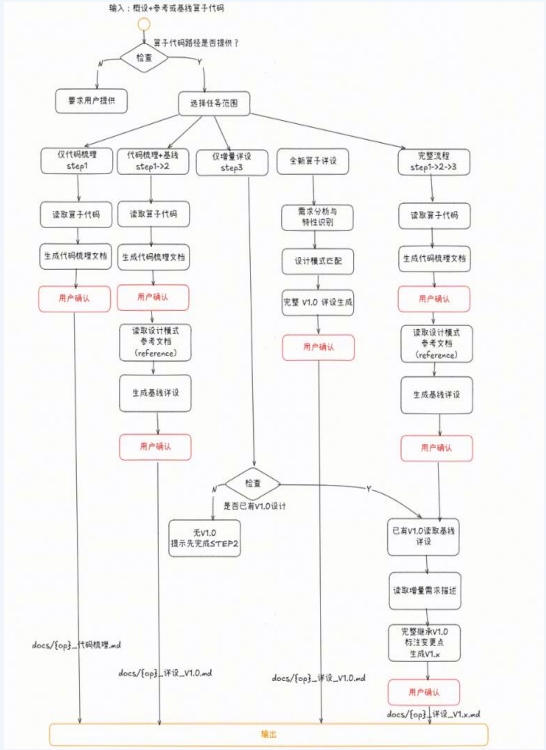
① 标准化设计模板
涵盖接口设计、模板设计、计算流图、buffer 设计等章节

② 详设自动生成 Skill
集成多种人类专家设计策略，从概设自动生成详设文档

③ 落地成效
生成的设计文档在10+ 项目落地使用



▪ 标准化设计模板



▪ 详设自动生成 Skill



▪ 集成设计知识



▪ 生成的设计文档

环节③：测试设计与测试用例生成

测试生成流程

概要设计文档 → 测试设计文档 → 测试用例生成 → 测试框架生成 → 测试执行 → 工程集成

实现路径（5 步流水线）

1. 基于概设获取测试设计输出
2. 基于测试点定制设计规则
3. 通过知识库识别算子风险点
4. 基于测试设计生成测试用例 CSV
5. 基于 CSV 和测试设计生成测试

关键挑战

- ! 因子多、参数量大 → 输入组合爆炸
- ! Tensor 类型参数 → 人工构造成本高
- ! 测试点多 → 脚本适配量大、维护困难

核心功能

- ★ **流程：** AI + 人工闭环测试
- ★ **用例：** 按类型/类别差异化生成
- ★ **框架：** CSV 与框架解耦

反向门禁（三层纠偏）

Layer 1 • 输入层纠偏
概设解析数据源符合测试设计输入获取标准

Layer 2 • 设计层纠偏
测试分析与用例生成无逻辑矛盾（测试点全面性、因子约束）

Layer 3 • 生成层纠偏
用例功能点数量、因子关系校验

落地效果

维度	输入依据	交付成果	检查
测试生成	设计 / 风险	用例 CSV	**覆盖**
框架生成	用例 / 接口	执行框架	**适配**
测试执行	测试集	验证记录	**回归**

核心结论：以规则驱动测试设计，用例与框架解耦；通过输入、设计、生成三层纠偏支撑验证闭环。

环节④ / ⑤：代码生成 + 实现审查

先规划后编码、逐层细化；编译门禁让问题在最小上下文时早暴露

规格提取+骨架	首用例设计	全量生成计划	Tiling实现	Kernel框架	Kernel内部	实现审查	定点返工	精度调试
spec-and-skeleton	test-designer	full-planner	tiling-coder	framework-coder	impl-coder	full-reviewer	full-reworker	debugger+curator
生成可编译骨架	基础用例	只读不写源码；三层计划带理由	OpDef/TilingData/TilingCheck/plugin	GM初始化/调度/同步TODO占位	Cube→Vec串行；共用common.h防撞车	交付质量验收，不进debugger兜底	只修P0/P1问题单，不重新接管	精度闭环→知识沉淀

正向建设思路

- ✓ 计划先行：三层计划（tiling/kernel框架/kernel内部）带"为什么"
- ✓ 骨架先行：编译前置，阶段零骨架必须可编译
- ✓ 测试先行：实现前golden与用例已就绪
- ✓ 接力棒 handoff：阶段二/三/四依次追加，后段不改前段
- ✓ 串行约束：Cube与Vec共用common.h，A编译过才进B

反向门禁：编译门禁（强制判据）

- ✓ 阶段零骨架构建通过 → 才进下游
- ✓ 阶段二 tiling构建通过 → 才进kernel实现
- ✓ 阶段三框架构建通过（TODO占位不影响）→ 才进kernel内部
- ✓ 阶段四 A(Cube)编译过→B(Vec)→全量构建过 → 才进审查

与增量流程的核心区别：每阶段编译

反向门禁：产物门禁 + 知识门禁

- ✓ 主调度只检查文件存在性，不读内容（检查成本极低且不越权）
- ✓ op_spec.json / case_plan.md / full_plan.md / CONTEXT.md /
- ✓ implementation_handoff.md 逐阶段核验
- ✓ 知识门禁：涉及API/Tiling/同步/硬件必须读知识库，
- ✓ 采用或拒绝记录到 knowledge_usage.md（可审计）

reviewer 审查 → **问题打回** → **reworker 定点修复** → **review_gate 门禁结论**

闭环规则

- ☐ reviewer发现问题打回reworker，不得直接回到coder
- ☐ 最多往返两轮；第二轮后仍有问题 → 未解决项作为风险写入门禁再进debugger，防止无限返工循环

reworker 定点纪律

- ☐ 只处理最新一轮P0/P1问题及必需同链路补丁，不重新接管full_plan
- ☐ 只做静态/编译级修复；正确性问题留给debugger
- ☐ 修改common.h契约必须留痕（IMPL-CHANGE注释+handoff+登记表）

环节⑥：精度调试—症状路由

痛点问题

① AICore 异常问题复杂

buffer 访问冲突、核内信号不匹配等

② 精度问题类型多

数据溢出、计算逻辑错误、未初始化、算法实现与公式不匹配等

③ 依赖专家知识&经验

模型缺乏领域专业知识，复杂问题的根因定位具有挑战性

Skill 路由



症状类别	目标 Skill	说明
崩溃	ascendc-coredump-disassemble	coredump + 反汇编定位崩溃位置
卡死	ascendc-sync-router（子路由）	委托子路由，内部再分核内/跨核
卡死	ascendc-sync	同步机制参考路由
卡死	ascendc-sync-debug	同步问题静态分析
卡死	ascendc-sync-crosscore-debug	跨核同步动态调试
卡死	ascendc-sync-incore-debug	核内同步静态分析
卡死	ascendc-sync-incore-topn-analysis	核内同步自动扫描
卡死	ascendc-sync-incore-tracker	核内同步动态追踪
内存	ascendc-debug-memcheck	mssanitizer 内存错误扫描
内存	ascendc-fault-injection	NaN 注入检测未初始化内存
精度	ascendc-precision-loop	自动迭代精度调试循环

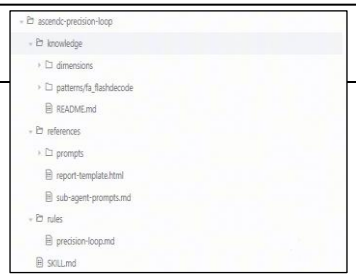
环节⑥：精度问题自闭环 — 如何做到“自动驾驶”

自动闭环机制：围绕精度问题开展「发现 → 定位 → 修复 → 验证」，保留运行证据与检查点。

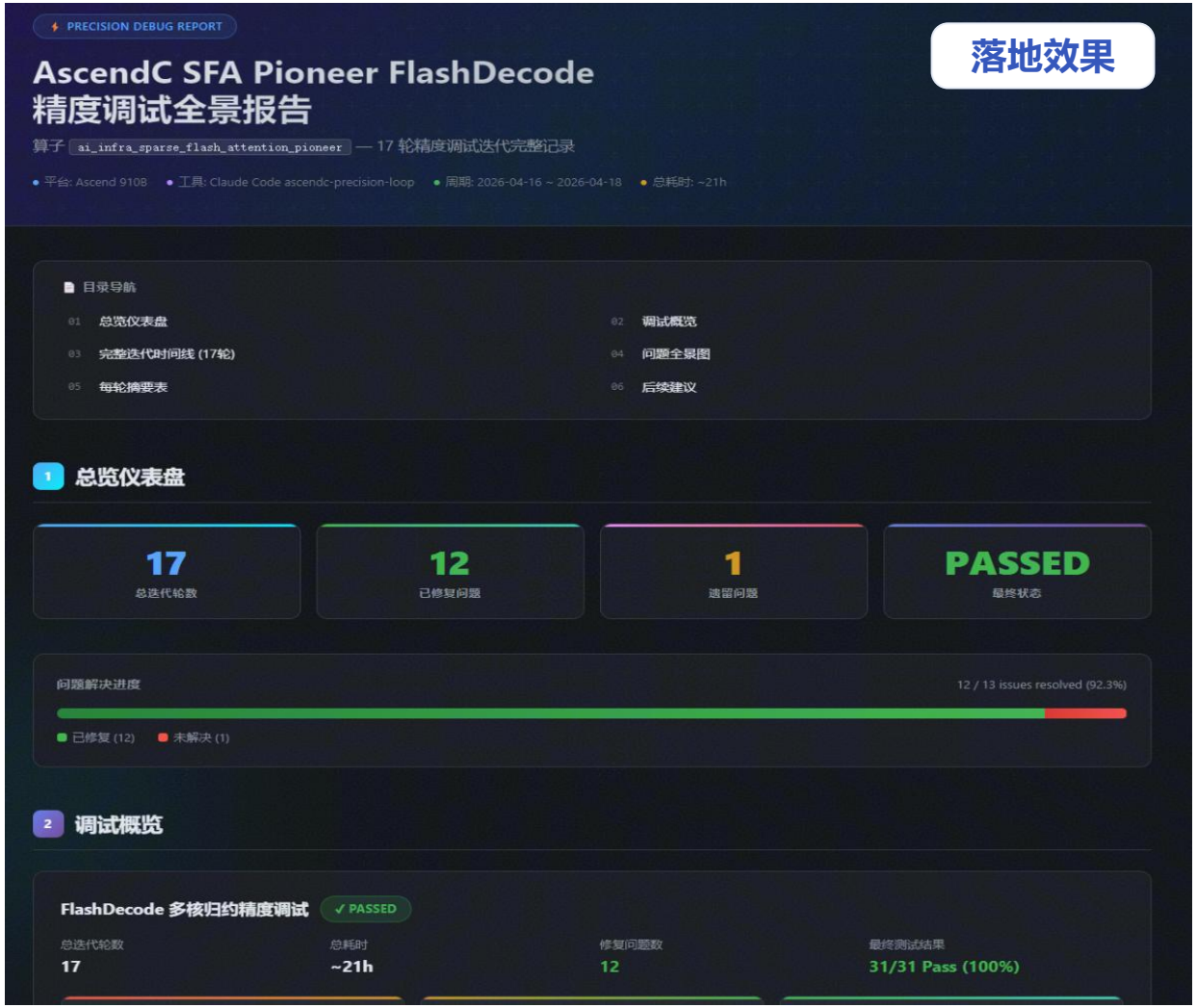
自动定位 SKILL

- **coredump-disassemble** — 反汇编定位
- **sync-router** — 核内 / 跨核
- **memcheck · fault-injection** — 内存扫描 + NaN 检测
- **precision-loop** — 自动迭代闭环

集成专家经验 & 知识



落地效果



实战：17轮迭代、31个测试用例验证，修复12个精度问题，覆盖内存异常、流水排布与代码逻辑。

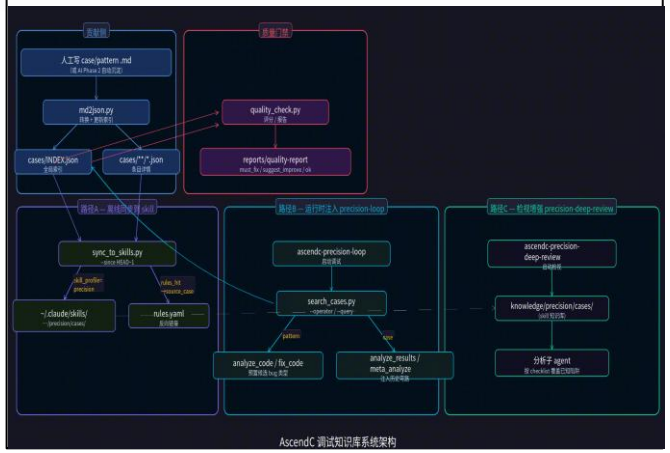
环节⑦：知识沉淀·结构化知识库·人与 AI 双向流转知识飞轮

- **理念**：把人工沉淀的经验和 AI 自动沉淀的经验放到同一个地方，让**人和 AI「都能读、都能写、都能用」**，打造**人与AI双向流转知识飞轮**
- **结构化知识库建设**：制定统一的结构化模板，集成调试案例、知识、设计策略等**100+**条
- **同步知识到关联SKILL**：新案例合入后同步到代码检视/问题定位/设计等Skill，打通更新链路，**知识增长了下游Skill同步变强**
- **正向飞轮**：人 / AI 踩坑 → 案例入库 → 人 / AI 定位更快，**团队与 AI** 的开发、调试能力在持续复用中同步提升

结构化共享知识库



人与AI双向知识流转飞轮



人/AI 踩坑 → 案例入库 → 人/AI 定位更快 → 团队能力同步提升

⟳ 人与AI双向知识流转飞轮：越用越好

落地效果：案例知识->增强AI代码检视能力



落地成果：从“个人提效工具”走向“组织基础设施”的跃迁

工具辅助（个人提效）→ 流程编排（团队协同）→ 知识沉淀（组织智能）→ 自演进（越用越强）

算子生成
16 个 FA 算子，覆盖简单、中等与复杂场景

16个
算子生成实践

详设生成
标准化模板 + 自动生成 Skill，10+ 项目落地

80%+
设计文档整体采纳率

质量保障
设计确认、阶段编译与运行回归

七环节
全过程门禁与验证证据

精度自动驾驶
发现 → 定位 → 修复 → 验证全自动闭环

12个
17 轮 / 31 用例 / 修复 12 个精度问题

知识库
专题 wiki 文档 120+，覆盖 11 个专项

100+
结构化案例/知识/策略，一键同步 Skill

共建共用
AscendSkills 随版本发布；CANNBot 集成

20+
Agent/Skills

赋能推广
AI 辅助研发分享

300+
10+ 次分享，覆盖 300+ 人次, 提升团队AI工程化能力

总结

✓ AI 辅助研发：把人的好纪律（设计先行 / 角色分离 / 证据先行 / 知识沉淀）自动化，把踩坑经验固化为可召回、可度量、可流转的知识资产，让团队与 AI 的开发、调试能力在持续复用中同步提升

✓ AI 辅助研发从“个人提效工具”走向“组织基础设施”的本质跃迁。从解决“单点问题”到“多 Skill 协同、多人协作、能力复用"

CANN AtomGit

THANKS



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



从EasyAsc到PyPTO Pro智能体

程禹

CANNBot Agent 技术专家



CANN AtomGit

00 方案介绍

Solution overview

CANN AtomGit

本次分享：第 1 名与第 3 名方案

硬件 全部 950PR 950DT A3 更多筛选 ⌄ 搜索方案、任务、算子或用户…

#	方案	硬件	用户	总分 ↓	平均分 ↕	基准加速比 ↕	理论加速比 ↕	提交算子 ↕	测例 ↕
01	CANNBot EasyAsc 450USD budget 106 任务 · official-tasks · 1.1	950PR	ddwe3 CANNBot	8591.76	81.05	1.659×	0.245×	53/53	99.3%
02	CANNBot_CAKE 104 任务 · official-tasks · 1.1	950PR	Deng_Pan	8335.17	78.63	1.334×	0.196×	53/53	98.6%
03	CANNBOT PYPTO 106 任务 · official-tasks · 1.1	950PR	ddwe7	8031.53	75.77	1.209×	0.171×	53/53	97.7%
04	CANNBot-Knowledge Ops Source Distillation 60 任务 · official-tasks · 1.1	950PR	qianbi1999 CANNBot-Knowl…	8013.23	75.60	1.072×	0.141×	53/53	96.5%
05	CANNBOT CannCraft 86 任务 · official-tasks · 1.1	950PR	qq_37913898	7738.43	73.00	0.976×	0.134×	53/53	97.8%
06	direct-invoke@Claude deepseek-v4-flash 94 任务 · official-tasks · 1.1	950PR	deepsleep	7523.74	70.98	0.825×	0.109×	53/53	96.8%
07	CANNBOT LINGXI-EVO 105 任务 · official-tasks · 1.1	950PR	yyzhang0715	7249.69	68.39	0.603×	0.079×	53/53	96.7%

CANN AtomGit

01 Easyasc

A kernel DSL for Ascend

第一部 · easyasc

01 Easyasc

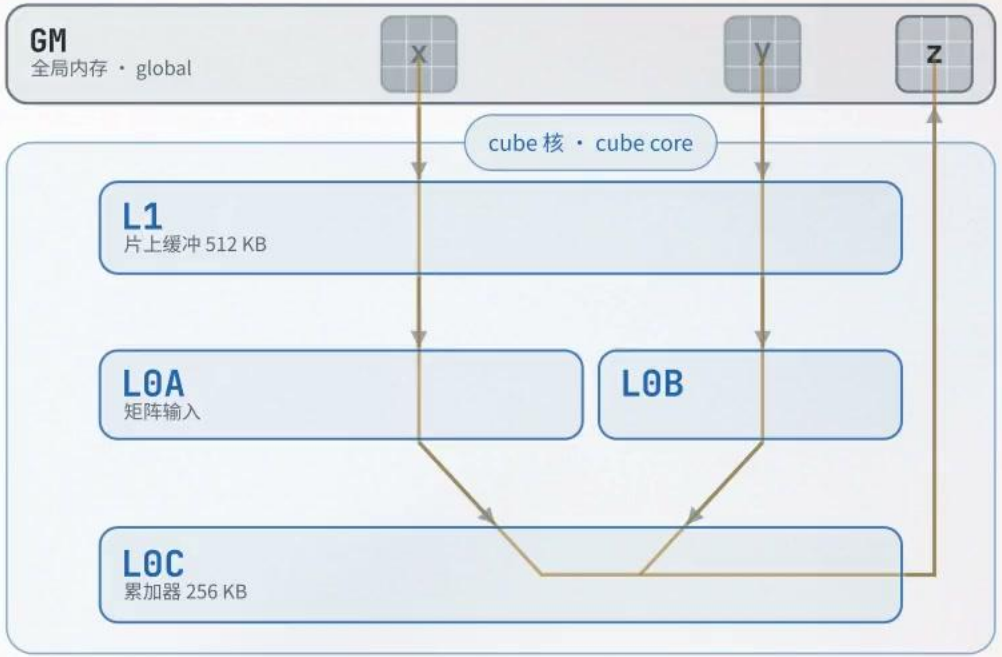
A kernel DSL for Ascend · 第一部 · easyasc

- 一个真实的 matmul kernel

kernels/950/matmul/matmul_float_mmad.py — 10 lines

```python
# Ascend 950

@kernel()
def matmul_float_mmad_kernel(
        x: GMTensor, y: GMTensor, z: GMTensor,
        M: Var, N: Var, K: Var):
    l1x = Tensor(DT.float, [M, K], Position.L1)
    l1y = Tensor(DT.float, [N, K], Position.L1)
    l0c = Tensor(DT.float, [M, N], Position.L0C)
    with auto_sync():
        l1x <<= x[16:16+M, :K]
        l1y <<= y[32:32+N, :K]
        matmul(l0c, l1x, l1y,
               m=M, n=N, k=K, is_init=True)
        z[16:16+M, 32:32+N] <<= l0c
    return z
```



第一原则　整条数据通路都写在代码里 · the whole datapath is in the code

简洁，清晰，易用

02 Cube / Vec 融合

One fused kernel · 第一部 · easyasc

fused_kernel.py · 950

```python
@kernel()
def fused(x: GMTensor, y: GMTensor, z: GMTensor, N: Var):
    lhs = Tensor(DT.half, [16, 64], Position.L1)
    rhs = Tensor(DT.half, [N, 64], Position.L1)
    acc = Tensor(DT.float, [16, N], Position.L0C)
    bridge = Tensor(DT.float, [8, N], Position.UB)
    output = Tensor(DT.float, [8, N], Position.UB)
    cv = CvMutex(0, src_end_pipe=Pipe.FIX,
                 dst_end_pipe=Pipe.V, depth=1)
    m = GetCubeIdx() * 16
    row = m + GetSubBlockIdx() * 8
    with auto_sync():
        lhs <<= x[m:m + 16, :]
        rhs <<= y[:, :]
        matmul(acc, lhs, rhs)
        cv.lock()
        bridge <<= acc
        cv.ready()
        cv.wait()
        epilogue(bridge, output, N)
        cv.free()
        z[row:row + 8, :] <<= output[:, :]
    return z
```

Vec · @vf()

```python
@vf()
def epilogue(src: Tensor, dst: Tensor, N: Var):
    reg = Reg(DT.float)
    for i in range(8 * N // 64):
        reg <<= src[i * 64]
        reg <<= reg.abs() + 1.0
        dst[i * 64] <<= reg
```

$$z = abs(x @ y.T) + 1$$

950 · X[512,64] · Y[64,64]

fp16 输入，fp32 输出

Cube：矩阵乘，L0C 写入 UB

Vec：abs 加 1，再写回 GM

同一个 @kernel 完成两侧计算

02 纯白盒模拟器

Instruction semantics in Python · 第一部 · easyasc

interleave 的交错规则

easyasc/simulator/pipe_micro.py

```
half = lane_count // 2
out0[0::2] = s0[:half]
out0[1::2] = s1[:half]
out1[0::2] = s0[half:]
out1[1::2] = s1[half:]
```

float32 · 每个寄存器 64 lane

```
s0 = [0,...,63]   s1 = [64,...,127]
out0 = [0,64,1,65,...,31,95]
out1 = [32,96,33,97,...,63,127]
```

agent 按需定位 Python 指令实现

减少 API 文档检索的 token 开销

模拟 trace，初步定位瓶颈

融合样例 · trace 分析

```
# fusion-trace.json
# core 0 / modeled cycles

pipe          active cycles
cube.MTE2               920
cube.M                   83
vec0.V                   80
vec0.MTE3               364

makespan               1786
timed events            448
```

查看搬运、计算与等待，缩小排查范围

同一份 kernel，可执行验证，也可导出 trace

模拟时序用于初步定位，最终性能以真机测量为准

CANN AtomGit

03 agent 的地图

router-first

第一部 · easyasc

03 agent 的地图

router-first · 第一部 · easyasc

• router-first

ROUTER.md

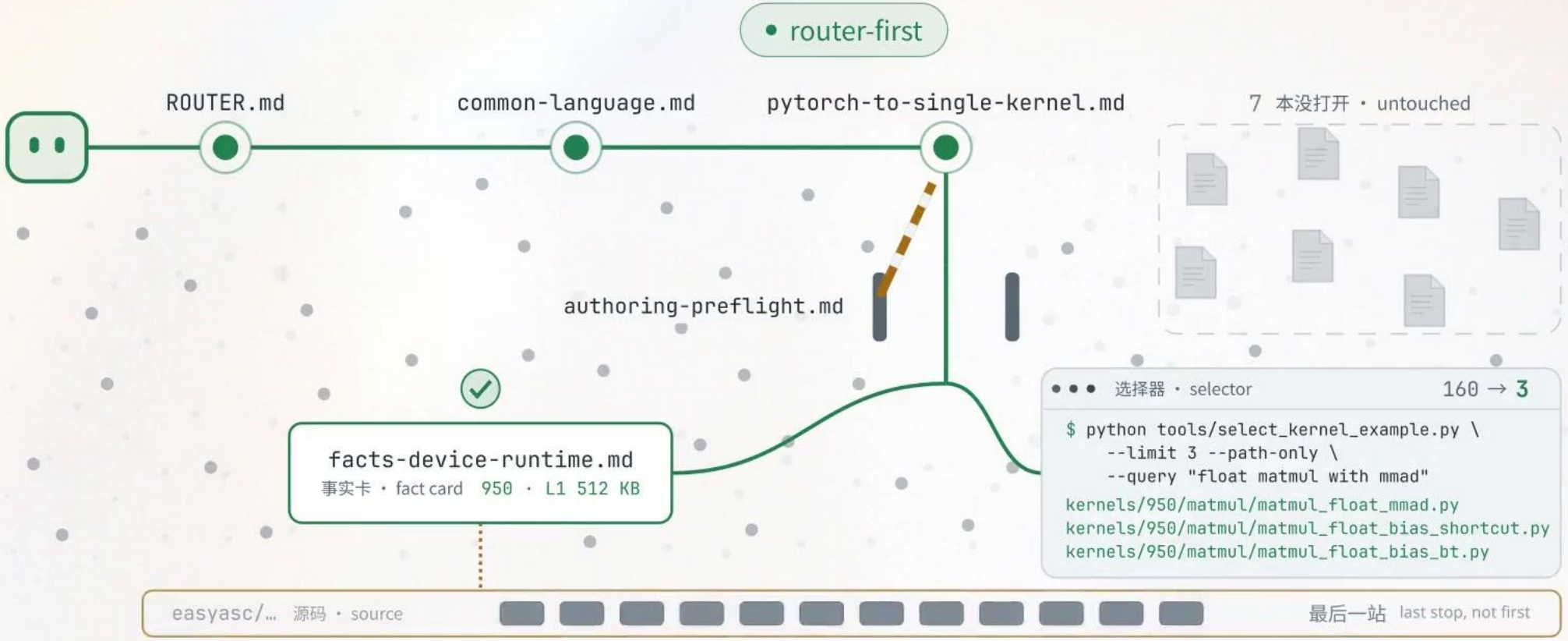
ROUTER.md · Workflow routes

AICPU kernel 编写 / 集成	aicpu-kernel-authoring
从 PyTorch / ONNX 写一个新 @kernel	pytorch-to-single-kernel ✓
把 PyTorch 函数拆成多个 kernel	decompose-pytorch
按已验证的分解实现 kernel	author-kernel-from-decomposition
调试已有 kernel	kernel-debugging
优化已正确的 kernel	kernel-optimization
CANN Bench 集成 / 评测 / 提交	cann-bench-aclnn-evaluation
改框架 / 测试 / 工具 / 文档	repository-maintenance

"Use the first matching row"

03 agent 的地图

router-first · 第一部 · easyasc



03

agent 的地图

router-first · 第一部 · easyasc



每一格里面，装着什么？

what is inside each stop?

ROUTER 与 preflight 上一幕已经走过 · seen in the previous act

03

agent 的地图

router-first · 第一部 · easyasc

• agent/ 的文档架 · the shelf

ROUTER | common-language | playbook | preflight | facts | constraints | patterns | pitfalls | diary

每一格里面，装着什么？

what is inside each stop?

ROUTER 与 preflight 上一幕已经走过 · seen in the previous act

03 agent 的地图

router-first · 第一部 · easyasc



03 agent 的地图

router-first · 第一部 · easyasc

• agent/ 的文档架 · the shelf

ROUTER | common-language | playbook | preflight | facts | constraints | patterns | pitfalls | diary

agent/playbooks/pytorch-to-single-kernel.md

流程与判据 / procedure & criteria

执行流程

01 读契约 → 02 选参考 → 03 最小探针 → 04 验证候选 → 05 交付或停止

完成判据 1 个 @kernel + 1 个 OpExec，且验证通过

停止预算 最多 3 个候选，同一失败边界最多 2 次修复

03 agent 的地图

router-first · 第一部 · easyasc



03 agent 的地图

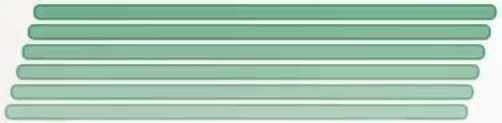
router-first · 第一部 · easyasc

• 按任务选取上下文 · task context

ROUTER | common-language | playbook | preflight | facts | constraints | patterns | pitfalls | diary

一次任务真正读入
what one task actually loads

≈ 36 KB



六份文档
six documents

ROUTER.md	3.8 KB
common-language.md	10.7 KB
playbooks/pytorch-to-single-kernel.md	5.0 KB
references/authoring-preflight.md	4.3 KB
references/facts-device-runtime.md	8.5 KB
references/pattern-index.md	3.5 KB
合计 total	35.9 KB

03 agent 的地图

router-first · 第一部 · easyasc

• 按任务选取上下文 · task context

ROUTER | common-language | playbook | preflight | facts | constraints | patterns | pitfalls | diary

一次任务真正读入
what one task actually loads

≈ 36 KB



六份文档
six documents

ROUTER.md	3.8 KB
common-language.md	10.7 KB
playbooks/pytorch-to-single-kernel.md	5.0 KB
references/authoring-preflight.md	4.3 KB
references/facts-device-runtime.md	8.5 KB
references/pattern-index.md	3.5 KB
合计 total	35.9 KB

03 agent 的地图

router-first · 第一部 · easyasc



剩下的知识，留在原地待命

the rest stays where it is, on call

04 编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

• playbook · § Work loop

pytorch-to-single-kernel.md

固定交付
fixed outcome

@kernel

OpExec

workflow.json count 0 · fixes 0



tools/select_kernel_example.py 160 → 3

```
$ python tools/select_kernel_example.py \
    --limit 3 --path-only \
    --query "float matmul with mmad"
kernels/950/matmul/matmul_float_mmad.py
kernels/950/matmul/matmul_float_bias_shortcut.py
kernels/950/matmul/matmul_float_bias_bt.py
```

• playbook · § Work loop

pytorch-to-single-kernel.md

固定交付
fixed outcome

@kernel

OpExec

workflow.json count 0 · fixes 0



最少被证明的边界

当前实现中，证据最少的 DSL 或硬件能力

例：尾块搬运会越界吗？UB 布局符合指令要求吗？

先做对齐与尾块探针，再写完整候选

• playbook · § Work loop

pytorch-to-single-kernel.md

固定交付
fixed outcome

@kernel

OpExec

workflow.json count 0 · fixes 0



最少被证明的边界

当前实现中，证据最少的 DSL 或硬件能力

例：尾块搬运会越界吗？UB 布局符合指令要求吗？

先做对齐与尾块探针，再写完整候选

04 编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

• playbook · § Work loop

pytorch-to-single-kernel.md

固定交付
fixed outcome

@kernel

OpExec

workflow.json count 0 · fixes 0



@kernel #1

模拟器
OpExec(fn, simulator=True)

kernel 输出
candidate

vs

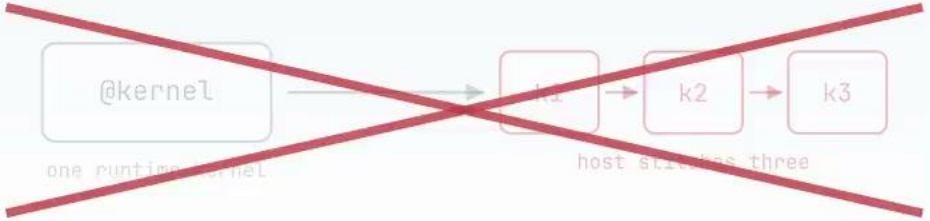
x @ y.t()
PyTorch reference

04 编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

NEVER · 偷偷改契约

把「一个 kernel」偷偷改成「host 拼三段」



Never relabel "one Python entry point" as "one runtime kernel".

playbook · Stop and report



DO THIS · 停下并报告

候选 3/3 | 修复 2/2

停下并报告 Stop and report

- 推断的契约 the inferred contract ✓
- 参考与证据 references, path/line ✓
- 候选 3/3 candidate count and paths ✓
- 验证证据 exact validation evidence ✓

第一个没解决的边界 the first unresolved boundary

04 编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

agent/ROUTER.md pick exactly one workflow

新写一个 runtime kernel	pytorch-to-single-kernel.md
把 PyTorch 拆成多个 kernel	decompose-pytorch.md
调一个坏掉的 kernel	kernel-debugging.md
让一个已经正确的 kernel 更快 ✓	kernel-optimization.md
搬到 PyPTO Pro 上跑	pypto-port.md
改框架 / 工具 / 文档	repository-maintenance.md

agent/playbooks/

kernel-optimization.md

```
## Optimization Target
## Entry Protocol
## Baseline Record
## Symptom Routing
## Reduction-Split Merge Gate
## Plateau Protocol — local vs structural levers
## Workflow
## C->V->C Lookahead Checklist
## Verification Rules
## Rollback Rules
## Reporting
## Provenance
```

前提：kernel 已经是对的

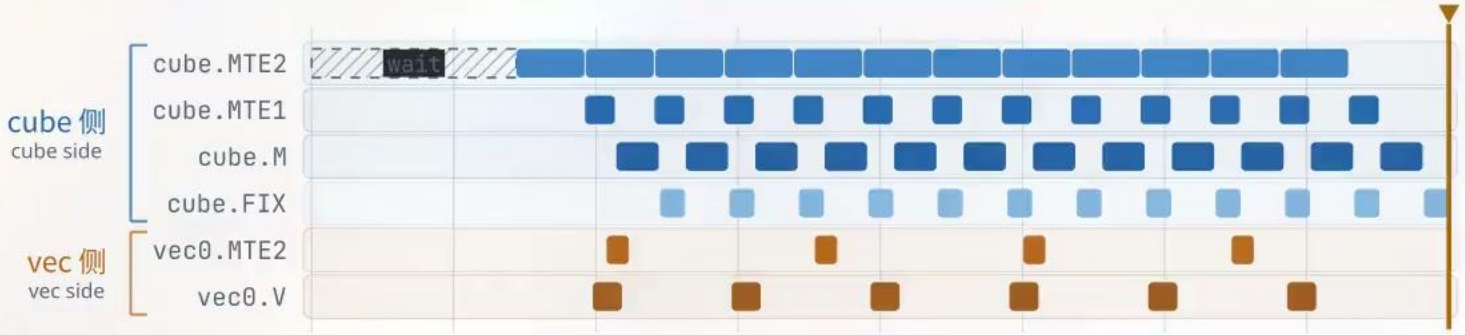
precondition: the kernel passes its reference check

04

编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

• kernel-optimization.md · Optimization Target



makespan = max(ts + dur)

over Chrome trace events
where ph == "X"

baseline

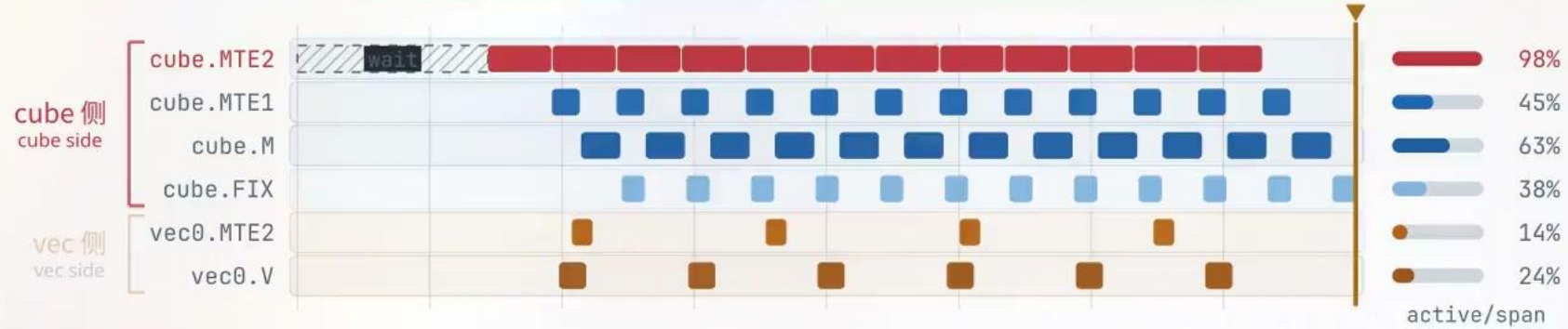
412800

cycles

04 编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

● kernel-optimization.md · Baseline Record · analyze_sim_trace



makespan = max(ts + dur)

over Chrome trace events where ph == "X"

baseline

412800

cycles

终端 · terminal

```
$ python tools/analyze_sim_trace.py tmp/opt/trace.json --group-by lane-pipe
Trace: tmp/opt/trace.json
Timed events: 18432
Time unit: cycles
Total time taken: 412800 cycles

Average utilization by pipe across cores
```

event count and occupancy illustrative · 事件数与占用为示意值

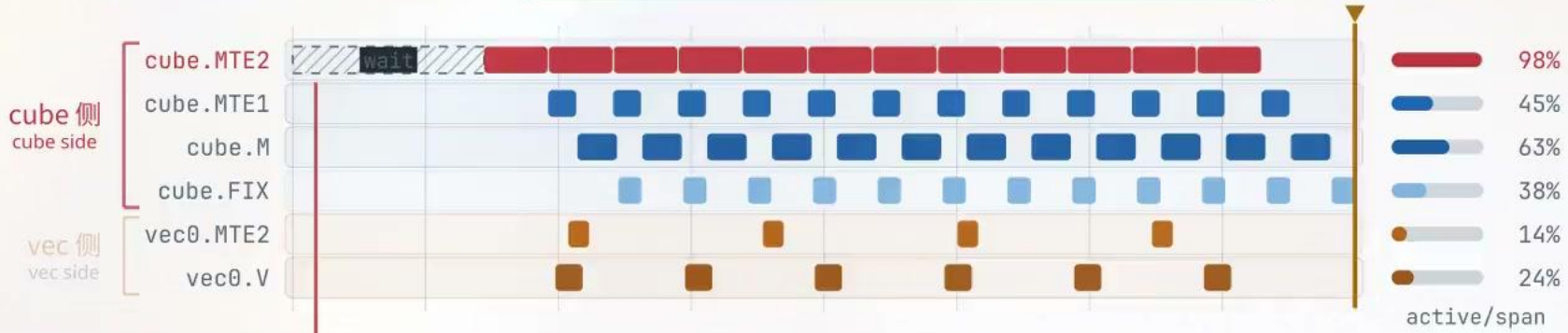
工具的回答 fast questions

1. 最忙的 pipe cube.MTE2 98%
2. 瓶颈在 cube 侧 不在 vec 侧
3. MTE2 前面堆着 wait

04 编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

● kernel-optimization.md · Symptom Routing



makespan = max(ts + dur)

over Chrome trace events where ph == "X"

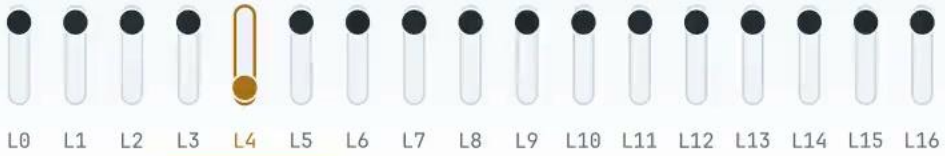
baseline

412800

cycles

杠杆 L0–L16 seventeen levers

agent/references/optimization/levers.md



L4 收紧同步

症状路由 Symptom Routing

trace 被 wait 和 barrier 主导

Trace is dominated by waits, barriers, or suspicious handoff placement

↓

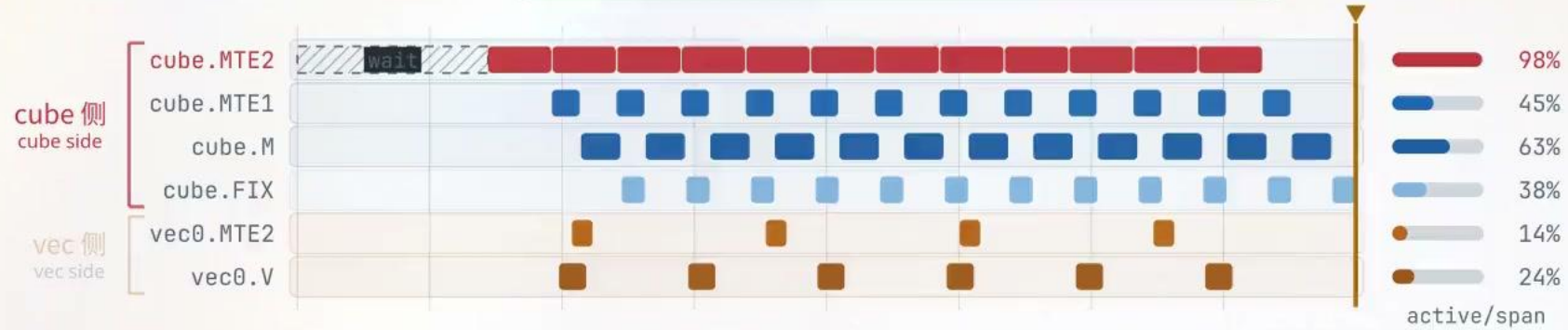
L4 收紧同步 Tighten synchronization to real lifetimes

一次只动一根 one lever at a time

04 编写与优化

Author a kernel, then make it fast · 第一部 · easyasc

● kernel-optimization.md · Plateau Protocol



makespan = max(ts + dur)

over Chrome trace events where ph == "X"

baseline

412800

cycles

只看占用 occupancy alone

✕ 还不是墙 not yet a wall

MTE2 98%

同一批字节被读了好几遍

that traffic is still reducible

"MTE2 is at 98%" alone does not;

占用 + 流量不可再减 occupancy + irreducible traffic

✓ 这才是墙 that is a wall

MTE2 98%

每个字节恰好读一次

every byte it moves is read exactly once

"MTE2 is at 98% and every byte it moves is read exactly once" does.


easyasc
ascriptor

05 ascriptor

Architecture and intermediate representation · 第二部 · ascriptor

matmul_float_mmad.py EP.01

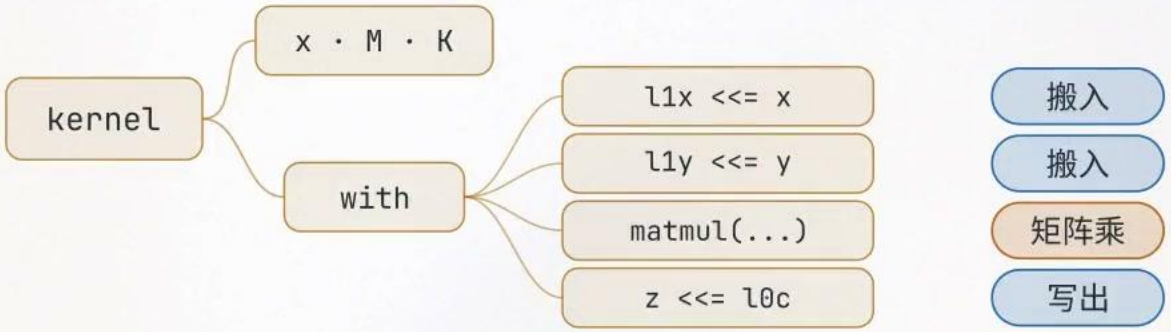
```python
@kernel()
def matmul_float_mmad_kernel(
        x: GMTensor, ...,
        M: Var, N: Var, K: Var):
    ...
    with auto_sync():
        l1x <<= x[:, :]
        l1y <<= y[:, :]
        matmul(l0c, l1x, l1y, ...)
        z[:, :] <<= l0c
    return z
```

• 同一段 Python · 两种生成方式

BEFORE • easyasc 先跑一遍，沿途记录 • 运行时收集

搬入 搬入 矩阵乘 写出

NOW • ascriptor 先读源码，再编成指令 • 读取源码后编译



06 多后端建立在共同表示上

Backends on a shared representation

第二部 · ascriptor

06

多后端建立在共同表示上

Backends on a shared representation · 第二部 · ascriptor

• ascriptor · 一份计算，多条生成路线



06

多后端建立在共同表示上

Backends on a shared representation · 第二部 · ascriptor

• ascriptor · 解耦后的分工

前端
读代码



共同的 IR 与改写
安排内存与同步



后端
生成代码

- CCE
- PTO ISA
- PyPTO Pro
- +

启动器
安排运行

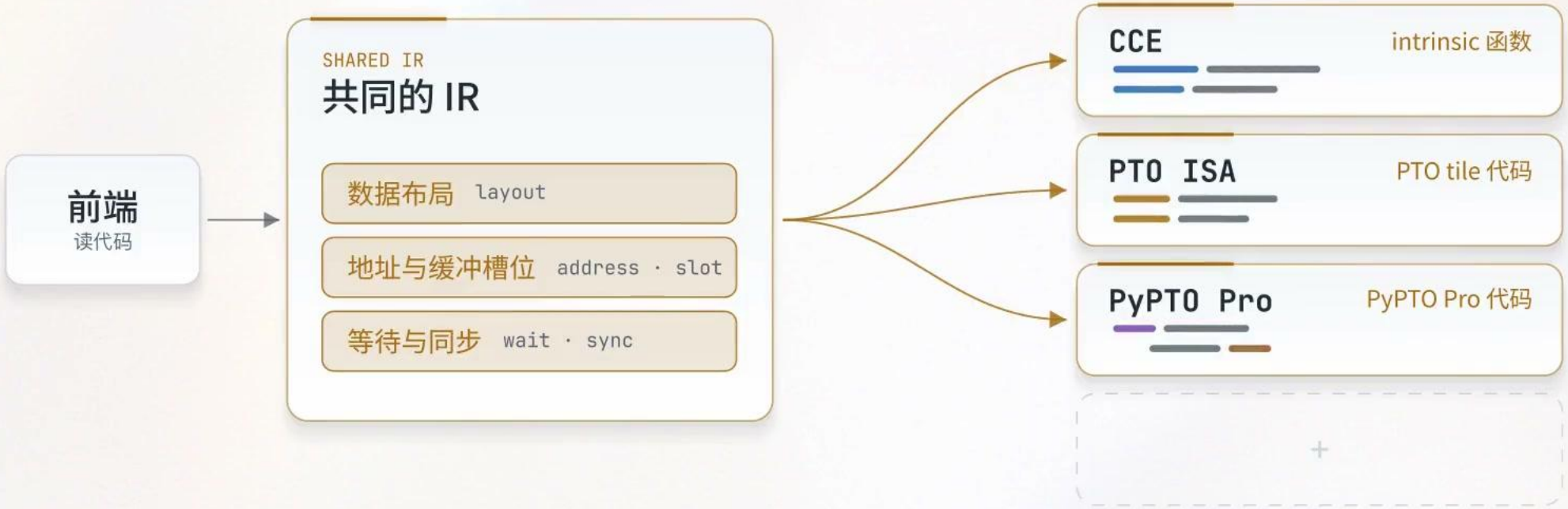
- aclnn
- cannsim
- board

解耦 各自负责，通过共同表示交接

06 多后端建立在共同表示上

Backends on a shared representation · 第二部 · ascriptor

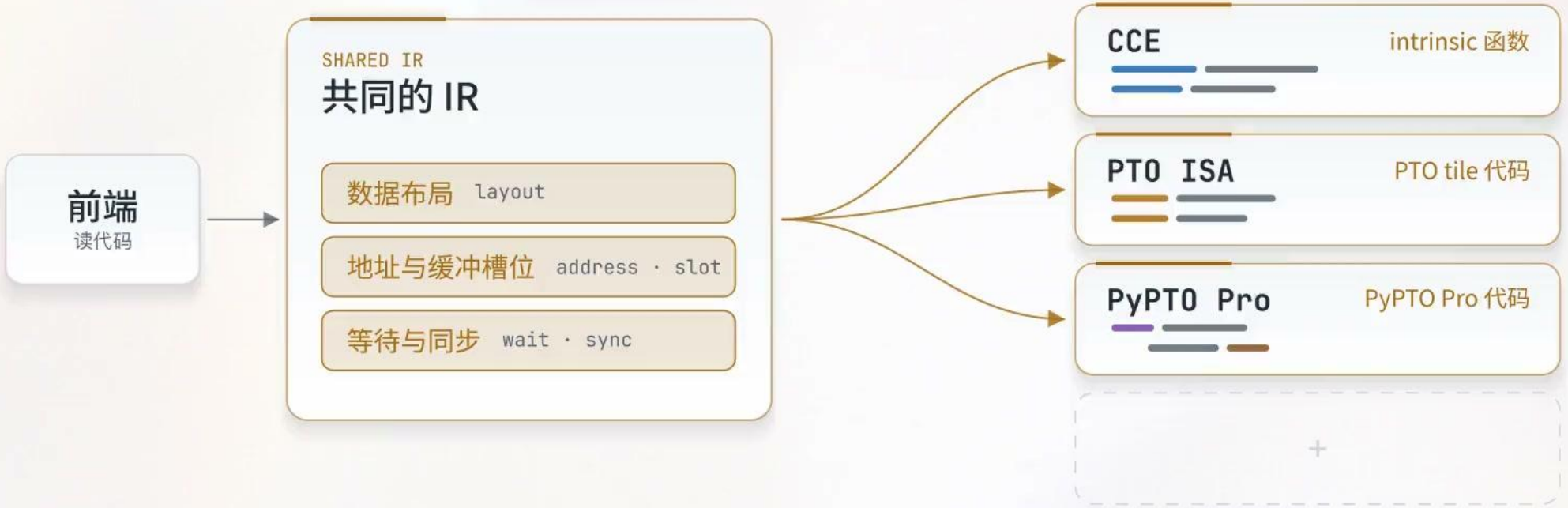
• ascriptor · IR 是共同的交接点



06 多后端建立在共同表示上

Backends on a shared representation · 第二部 · ascriptor

• ascriptor · IR 是共同的交接点



07

例子与起步指南

Units and a startup kit · 第二部 · ascriptor

07

例子与起步指南

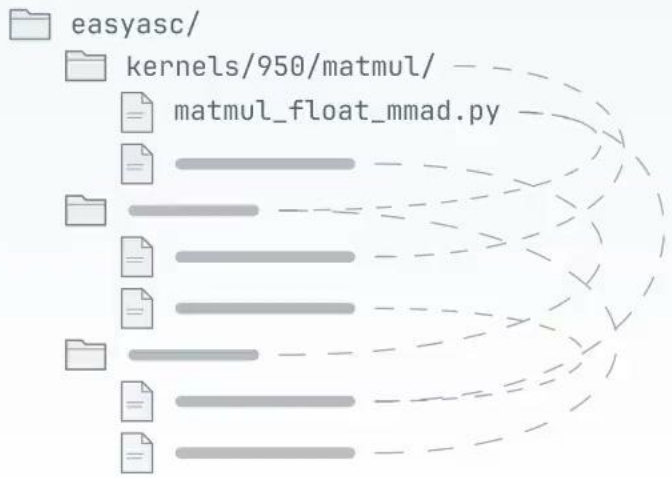
Units and a startup kit · 第二部 · ascriptor

BEFORE • easyasc

NOW • ascriptor

EASYASC

旧仓库里的例子



KERNELS

一个文件夹，一个单元



07 例子与起步指南

Units and a startup kit · 第二部 · ascriptor

• ascriptor · kernels 里的一个单元

KERNELS

一个文件夹，一个单元



UNIT

axpy/

- kernel.py — 计算
- reference.py — 参考答案
- main.py — 运行入口
- metadata.json — 适用说明

07 例子与起步指南

Units and a startup kit · 第二部 · ascriptor

BEFORE · easyasc



上下文 6 / 6

6 份，约 36 KB

NOW · ascriptor

首条消息 · first message

开始一个新任务

起步指南 20 KB

kernel-authoring.zh-CN.md

上下文

1 页，约 20 KB

08 拿任务检验指南

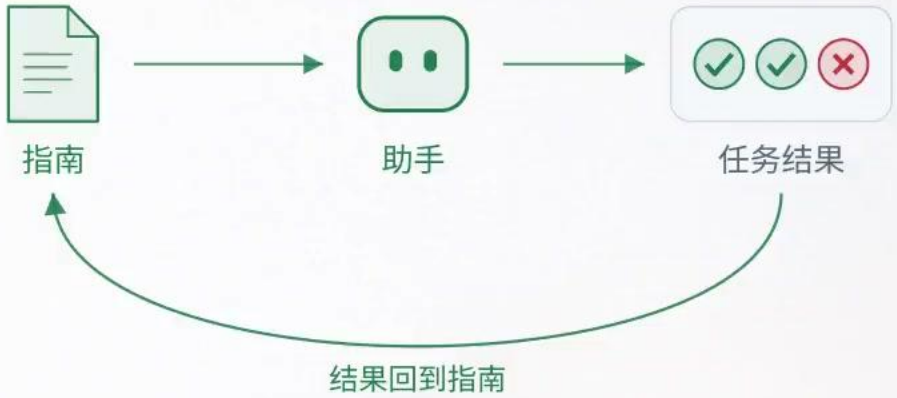
Measuring whether the guide helps · 第二部 · ascriptor

BEFORE • easyasc



告诉助手怎么做

NOW • ascriptor



检验它有没有帮上忙

CANN AtomGit

09

第三部 · PyptoPro AgentKit

PyPTO Pro Agent

Native kernels with an agent

09

PyPTO Pro Agent

Native kernels with an agent · 第三部 · PyptoPro AgentKit

AgentKit

kernel.py

```python
@pl.jit
def kernel(a, b, idx, out):
    c = pl.gemm(a, b)
    act(c, t)
    pl.simt.gather(idx, t, out)

@pl.vector_function
def act(c, t):
    t = vf.exp(c)
```



昇腾 NPU
Ascend NPU

CANN AtomGit

10 知识、样例与检查

Knowledge, examples and checks

第三部 · PyptoPro AgentKit

10 知识、样例与检查

Knowledge, examples and checks · 第三部 · PyptoPro AgentKit

AgentKit

kernel.py

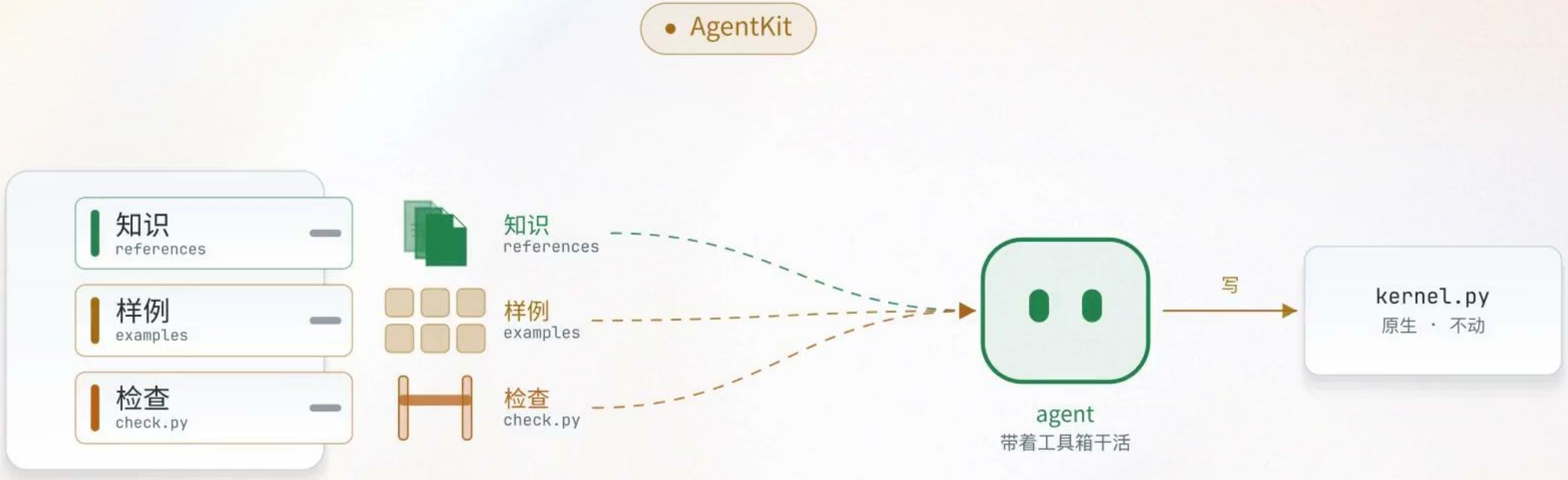
```python
@pl.jit
def kernel(a, b, out):
    c = pl.gemm(a, b)
    act(c, out)
```

- 知识 references
- 样例 examples
- 检查 check.py

编译 compile → 运行 run → 对账 reconcile ✓

10 知识、样例与检查

Knowledge, examples and checks · 第三部 · PyptoPro AgentKit



10 知识、样例与检查

Knowledge, examples and checks · 第三部 · PyptoPro AgentKit

• AgentKit · 知识



AgentKit

ENTRY · 入口

SKILL.md

该做什么		去哪查
实现一个 kernel	→	authoring
修编译错误	→	debugging
要拿设备时间	→	native-timing
优化一个对的 kernel	→	optimization
交付算子包	→	delivery
查 API 参数	→	api/*.pyi

workflows

debugging.md

症状 → 先查什么

这条错误 → 归谁管

知识给出下一步，样例展示 API 怎么用

10 知识、样例与检查

Knowledge, examples and checks · 第三部 · PyptoPro AgentKit



check.py：反查 API 样例，检查 DSL 合法性（静态规则）

10 知识、样例与检查

Knowledge, examples and checks · 第三部 · PyptoPro AgentKit

AgentKit

• AgentKit · 样例

check.py surface

```
vf.load_align
```

```
vf_reduce/kernel.py
x = vf.load_align(src)
```

```
vf_softmax/kernel.py
v = vf.load_align(row)
```

```
vf_arith/kernel.py
a = vf.load_align(lhs)
```

row_topk
grouped_gemm_swiglu_quant
layernorm
masked_softmax
moe_weighted_combine
paged_gqa_decode
quant_gemm_w8a16
embedding_bag
fused_adamw
gemm_rmsnorm
byte_histogram

check.py：反查 API 样例，检查 DSL 合法性（静态规则）

10 知识、样例与检查

Knowledge, examples and checks · 第三部 · PyptoPro AgentKit

• AgentKit · 样例



AgentKit

row_scale.py

```
1  @pl.vector_function
2  def row_scale(dst, src):
3      v = vf.load_align(src)
4      y = vf.astype(v, dst)
5      vf.store_align(dst, y)
```

STATIC GATE

check.py rules

- 地址 address
- 对齐 alignment
- 禁用写法 banned form

check.py：反查 API 样例，检查 DSL 合法性（静态规则）

10 知识、样例与检查

Knowledge, examples and checks · 第三部 · PyptoPro AgentKit

• AgentKit · 样例



AgentKit

文档纠正不了的，
交给确定性检查

新的限制会沉淀进检查器

RULES

check.py rules

- 地址 · address
- 对齐 · alignment
- 禁用写法 · banned form
- 名字出不了声明块 · stage 参数

+1

CANN AtomGit

11 实战

Real tasks

第三部 · PyptoPro AgentKit

• 会话实录 · 2026-10-12

cann-bench · level4

- gqa
- gru
- lstm
- mla
- sparse_flash_attention

TASK
grouped_matmul_swiglu_quant

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20

baseline · 公开基线

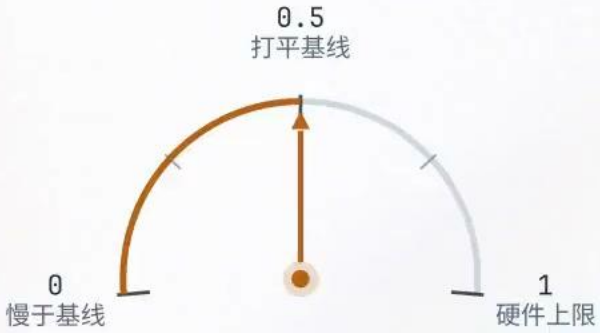
接下来看看实战中的表现

• 会话实录 · 2026-10-12



打平基线 25 / 50

逼近上限 50 / 50



• 会话实录 · 2026-10-12

参数扫描 · sub-1



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20

75.84

75.86

当前配置已是扫描空间内最优

72 分钟

11 实战

Real tasks · 第三部 · PyptoPro AgentKit

• 会话实录 · 2026-10-12

13:50 — 15:32

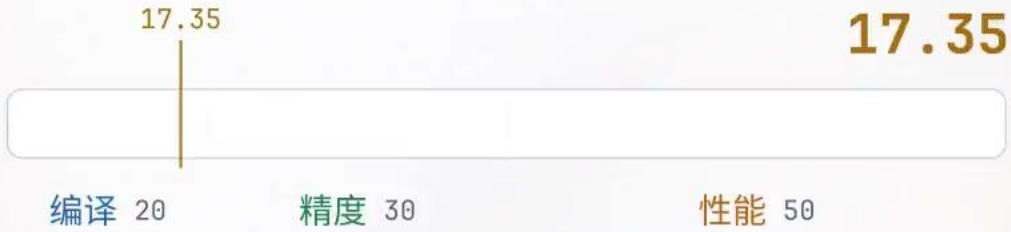
sparse_flash_attention

cann-bench · level4



开工时的官方成绩
the score on record

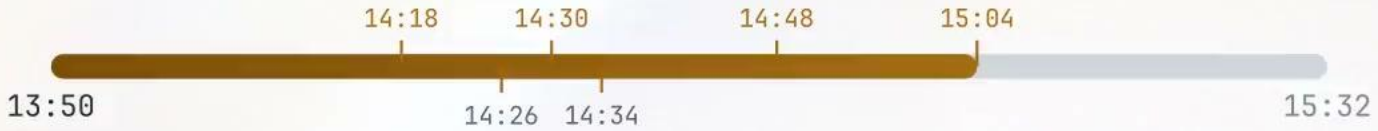
17.35



11 实战

Real tasks · 第三部 · PyptoPro AgentKit



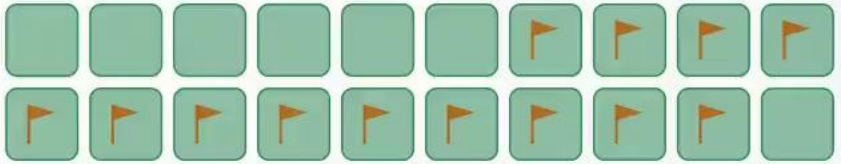


• 会话实录 · 2026-10-12



• 最高 5.78x

官方口径的加速比



81.40



编译 20　精度 30　性能 50

• 会话实录 · 2026-10-12

102 分钟

preload

块大小

+3

constraints.md

B
A

A/B 无可测变化

• 已回滚

11 实战

Real tasks · 第三部 · PyptoPro AgentKit

• 会话实录 · 2026-10-10

TASK
mla

69.11



编译 20　精度 30　性能 50

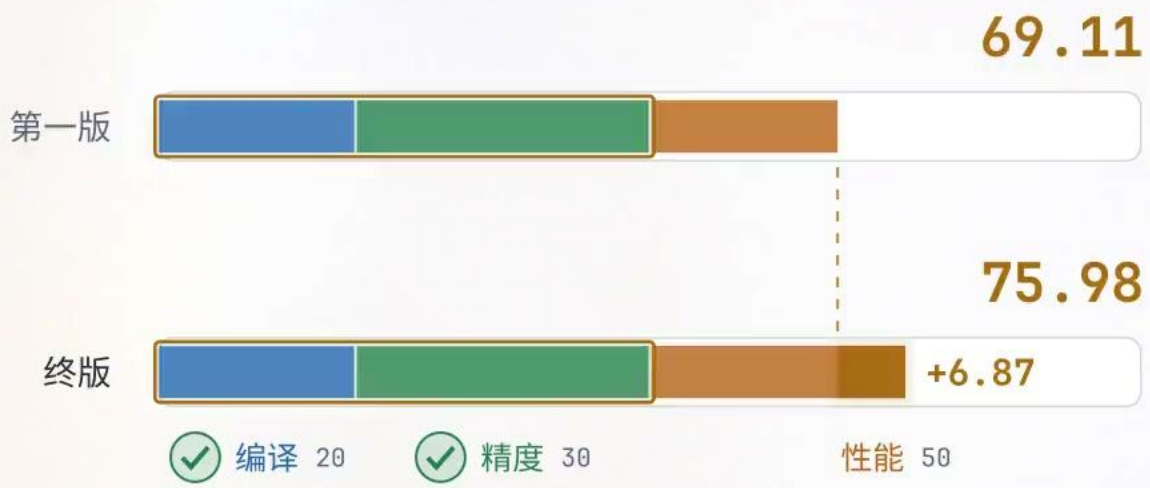


四例代表性用例时间 · us

11 实战

Real tasks · 第三部 · PyptoPro AgentKit

• 会话实录 · 2026-10-10

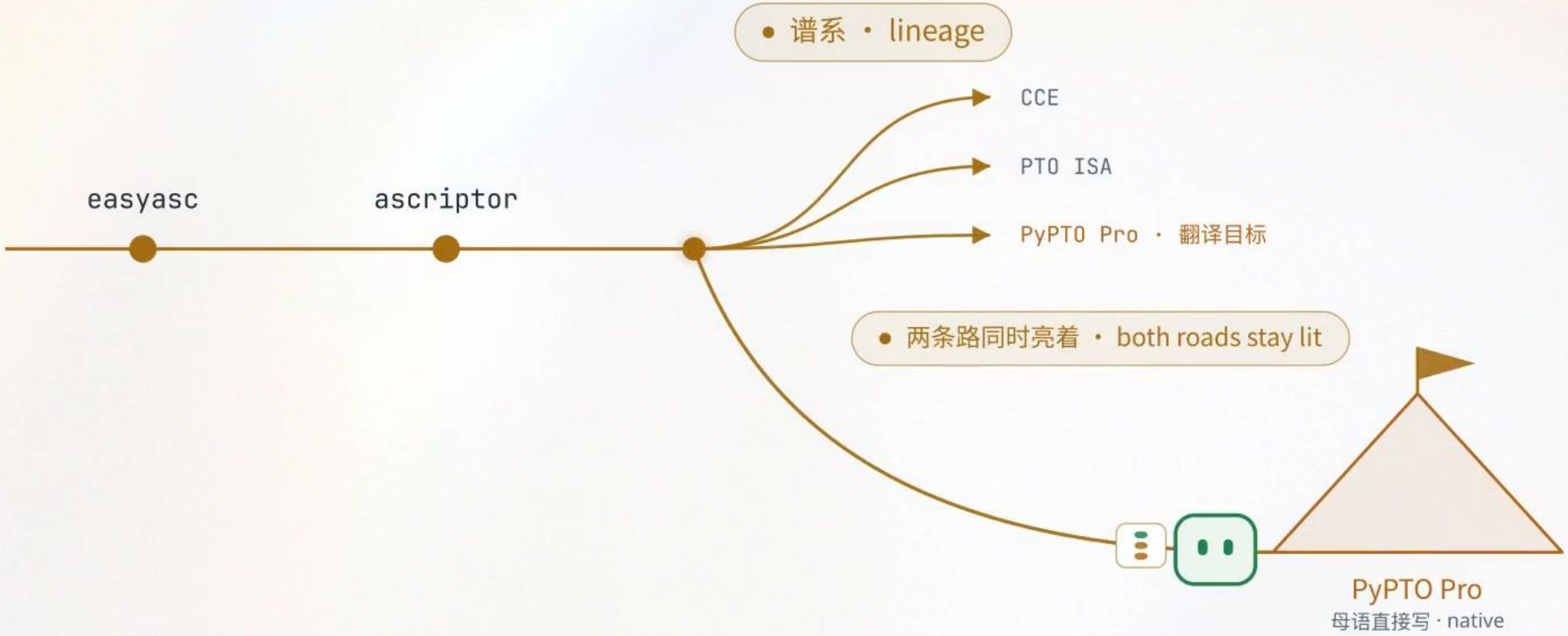


先算对，再提速

12

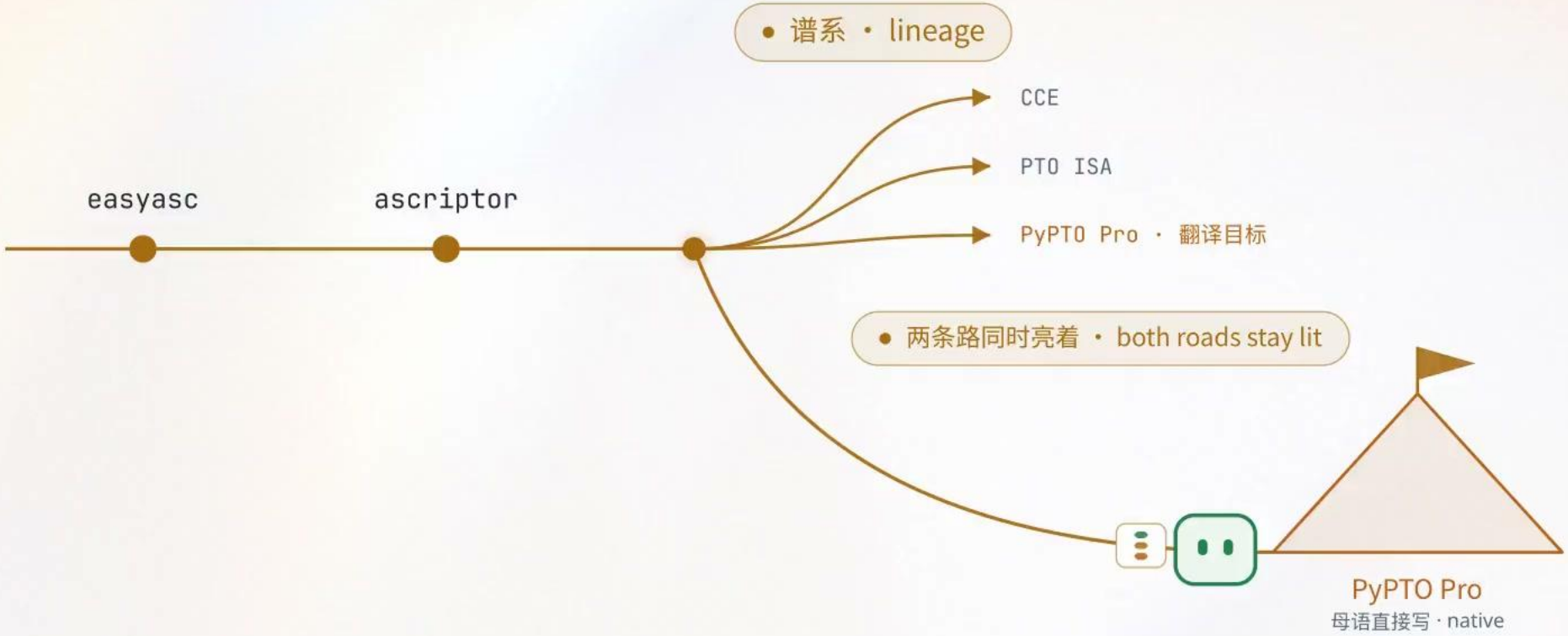
结语

A workflow built on reusable experience



12 结语

A workflow built on reusable experience



CANN AtomGit

THANKS



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



基于LIGNXI-EVO的算子生成性能调优Agent构建

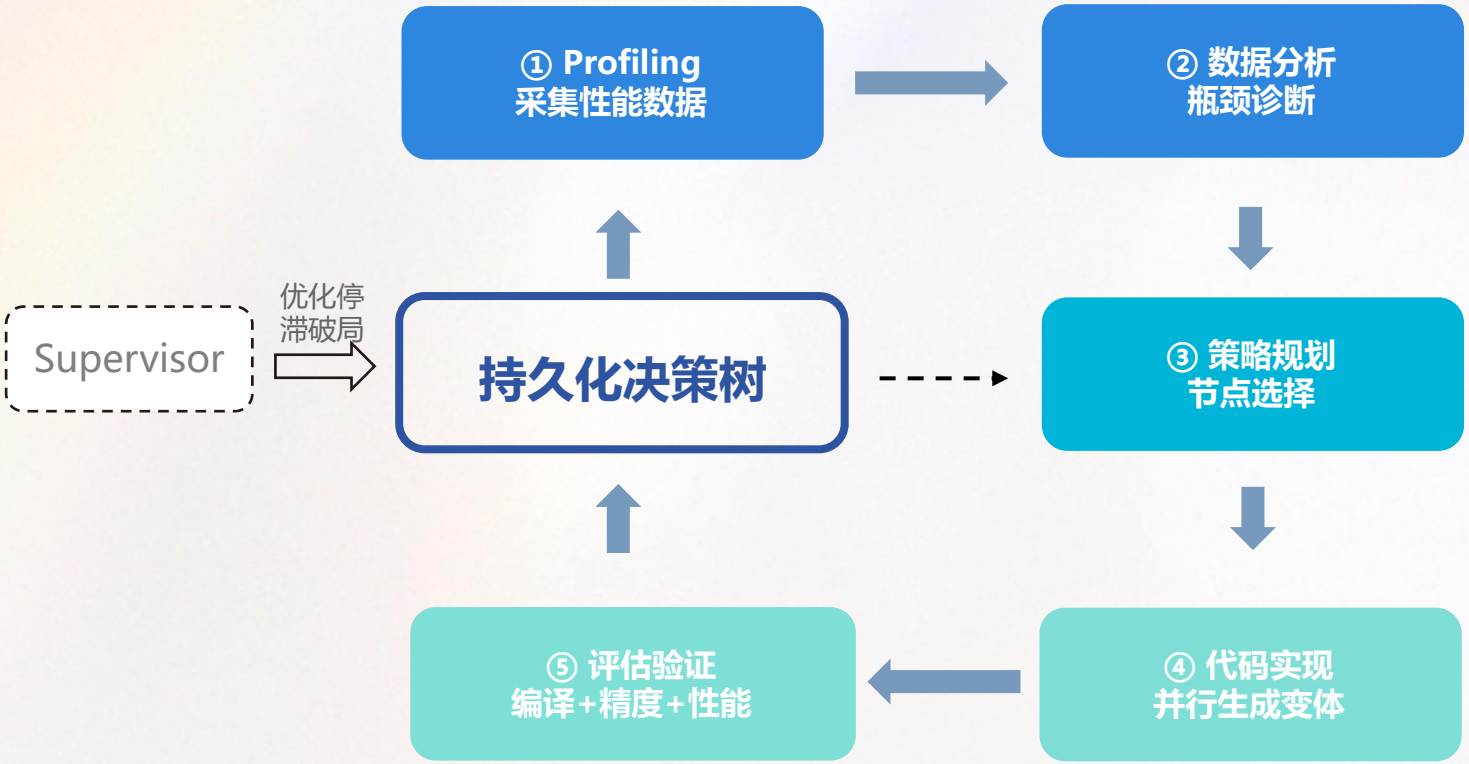
张奕裕

iCode团队 技术专家



CANN AtomGit

LIGNXI-EVO方案总览



每一步都有 skill 承载

① Profiling 采集 → ascendc-profiling

② 瓶颈诊断 → ascendc-profiling-analysis

③ 策略规划 → evolution-strategies

④ 落实实现 → playbook + subagent

⑤ 评估验证 → ascendc-evaluation

⑥ 决策树更新 → evolution-world-model

优化停滞 → 外部独立视角

连续无提升 → Supervisor 从外部知识 / 算法级 / 架构级审视，新方向插回树中

CANN AtomGit

核心设计：Kernel 优化 = 一次树上的搜索



搜索空间是显式的

每个节点 = 一个优化意图 + 实测结果

LLM 管树，不管细节

展开 / 剪枝 / 调置信度

规划与实现解耦

意图被否定，实现不推倒重来

CANN AtomGit

为什么这么做：实践中的两个发现

朴素 agent 的两个坑

① 缺乏规划

只看当前片段，**无法规划多步见效的路径**

例：先改布局、再上双缓冲——第一步单看是负收益，当场被否掉

② 对噪声敏感

一次编译错误 / 性能回退，**整个分支被抛弃**

失败里混着「意图错」与「实现错」，分不清，只能全丢

决策树给出的答案

坚持有前景的方向

方向写进树，不因一次失败丢掉

局部重试，不动意图

实现有缺陷只重试该节点的实现代码，上层意图保持不变

置信度跟随实际反馈

按编译日志、性能数据调整对各优化动作的置信度

插入假设 / 剪除分支

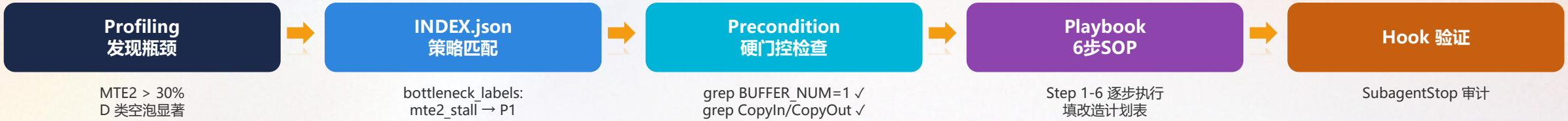
反馈驱动树的生长：加新假设，剪掉反复验证无效的分支

本质：把「一次成败」变成「树上一个节点的一次观测」——失败只是一次观测，不是一次判决。

CANN AtomGit

核心设计：策略知识形式化

➢ 以「双缓冲」策略为例



策略三件套

策略卡片Card

```
id: P1
name: 双缓冲优化
series: P (通用性能)
bottleneck_labels:
  [mte2_stall, fake_mte2_bound]
operator_families:
  [elementwise, reduction, ...]
conflicts: [P3(乒乓)]
synergy: [P10(向量化), P14(对齐)]
gain_avg: 1.15x
---
核心思路：将单缓冲改为双缓冲，利用流水线重叠掩盖
MTE2 数据搬运延迟。适用于 MTE2 占比>30% 的
memory_bound 算子。
```

前置检查Preconditions

```
# 三个硬门控，任一不满足 → 直接过滤，不进候选池
checks:
  - name: buffer_num_is_one
    grep: "BUF_NUM.*1[^0-9]"
    because: 只有单缓冲时双缓冲才有意义
  - name: has_dma_pipeline
    grep: "DataCopy|CopyIn"
    because: 无 DMA 流水线则无需掩盖延迟
  - name: not_pure_compute
    grep: "Mmad|Conv"
    because: 纯计算瓶颈应走 Compute-bound 路线
```

策略实操Playbook SOP

```
#6步SOP
Step 1: 定位缓冲区声明
  记录文件名:行号
Step 2: 填写改造计划表
  | 变量 | 当前 | 目标 |
Step 3: 识别缓冲区形态
  GM Buffer / TBuf / VecBuf
Step 4: 实施改造
  声明+初始化+同步点
Step 5: grep 自检 + Hook验证
  grep -c "BUF_NUM.*2“ → 结果必须 ≥ 1
Step 6: 陷阱提醒
  同步点必须同步增加！内存对齐 32B!
```

CANN AtomGit

核心设计：知识自闭环增长

① Prompt 动态组装 —— 不是写死的模板



② 知识闭环



能力随使用增长

策略库与世界模型随迭代单调扩充——每轮验证结论均固化为可复用资产，后续任务从更高的经验基线出发

正反例双向沉淀

通过验证的经验固化为策略卡与决策树节点；失败路径转化为约束条件，持续收窄搜索空间

沉淀全程可审计

每张策略卡自带来源、前置条件与实测记录——知识的产生与消费全程可追溯

CANN AtomGit

核心设计：防护栈，不信任 Agent 自报

PreToolUse(Bash) • 路径安全

空变量做 cp / mv / rm 路径参数 → 直接拦截

路径展开为 / 或 $HOME → 直接拦截

Stop • 主 agent 状态完整性

轮数没跑满 / 变体数不足 / 跳了 refine → 拦下，不许退出

缺评测产物、缺 msprof、诊断未写 → 先补跑再走

drift_status = replan_required → 先执行重规划

SubagentStop · 子 agent 审计

transcript 必须显示真的执行了 build / evaluate

implementation_note 必写；分到策略卡必须出现 Playbook 五步

判定只认文件事实证据

评测产物 · profiling CSV · 评测 JSON · Agent 日志等

自报状态一律覆写

LLM 写的 state 字段不采信，每次触发由钩子从磁盘重新推断

三问全过才算数

编译通过 · 精度对齐 · 性能提升；回写前区分「意图错 vs 实现错」——实现错只重试实现，意图错才剪枝。

CANN AtomGit

刷榜 Tips

1

环境对齐官方评测配置

本地 toolkit 与官方不一致（8.5.0 vs 9.1.0），同算子实现精度不同，曾误判精度不过；本地多为 910B、官方是 910C，性能精度均有微差——尽量贴近官方评测配置，结果才更准

2

用好官方 MCP / CLI 工具，全流程Agent自动化刷榜

早期人工提交看结果低效；接上 CANNBench MCP / CLI 后：下达任务 → 自动优化 → 自动提交 → 自动验证 → 未达标继续迭代，全循环无人值守

3

基线影响收敛速度 —— 优先用 ops 仓已有算子起步

基线太烂，每轮评估极慢、探索空间过大、无法快速收敛

- LINGXI-EVO已开源至CANNBot-Skills社区
- https://gitcode.com/cann/cannbot-skills/tree/master/plugins-community/ops-perf-evolution

CANN AtomGit

THANKS



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit



CANNBench Season2 发布

陈振宇

南京大学教授、深圳河套学院双聘教授



CANN AtomGit

新增 L5，拓展复杂任务评测

保留基础能力评测，进一步覆盖复杂模型计算与程序级任务

复杂任务的整体实现与性能优化

完整语义实现

处理多阶段依赖，保证输出与状态正确

整体性能优化

统筹计算分块、数据复用与阶段衔接

不同输入下的适应能力

覆盖任务约定的形状、类型与边界条件

主集合等级分布变化



L5 从 0 项增至 5 项，L4 从 8 项增至 13 项

Kimi Delta Attention

矩阵状态递推与跨 token 依赖

LACT TTT Chunk

分块计算与状态更新协同

Tensor Program VM

指令驱动的张量执行

调整前为原 53 项主线等级，调整后为 Season 2 规划等级，正式范围以发布版本为准

CANN AtomGit

主集合更新：调整目标与业界模型演进

结合真实模型计算需求，保留代表性任务、整合相近计算，补充新兴场景

本轮调整目标

跟进模型演进

吸纳主流模型与前沿架构中的关键计算
让评测任务与实际算子开发需求同步更新

优化场景覆盖

整合功能相近的任务
补充新型注意力、低精度量化
与跨层信息融合等计算场景

保持能力梯度

保留代表性基础算子
同时引入复杂模型计算
兼顾基础实现与综合优化能力

新增任务与业界模型演进

业界演进方向	代表性新增任务	本轮补充的计算场景
新型序列建模	Kimi Delta Attention（KDA）	跟进 Kimi 的门控 Delta 注意力机制 覆盖矩阵状态递推与跨 token 依赖计算
高效长上下文	Compressed Sparse Attention Core（CSA）	跟进压缩稀疏注意力 覆盖压缩 KV 与局部上下文联合参与的注意力计算
低精度计算	Dynamic MX Quant	顺应低精度计算的发展 补充块级 E8M0 共享尺度的 FP8 量化场景
跨层信息融合	Attention Residual	跟进残差连接的前沿演进 覆盖沿模型深度进行动态注意力聚合的计算

39 项保留 + 21 项新增 = 60 项

新增任务还涵盖训练梯度、序列卷积、稀疏访问与缓存、图像预处理等方向

CANN AtomGit

S1保留算子

L1

exp swi_glu

L2

apply_rotary_pos_emb
arg_max
cross_entropy_loss
cummin
dynamic_quant
gather
gcd
grid_sampler_3d
rms_norm
scatter
softmax
unsorted_segment_sum

L3

add_rms_norm_dynamic_quant
conv_2d
conv_3d_backprop_filter
dequant_swiglu_quant
depthwise_conv_2d
dilation_2d
engram_gate_fusion
grouped_matmul
mhc_sinkhorn
moe_gating_top_k_softmax
moe_re_routing
nms
quant_matmul
roi_align
strided_slice
top_k
transpose
unique
weight_quant_batch_matmul

L4

gqa
grouped_matmul_swiglu_quant
lstm
mla
mla_prolog
sparse_flash_attention

CANN AtomGit

S2新增算子

新增算子 01—11	Level
01 kimi_delta_attention	L5
02 compressed_sparse_attention_core	L5
03 flash_attention_score_grad_enhance	L4
04 sparse_lightning_indexer_grad_kl_loss_enhance	L5
05 attention_residual	L5
06 selective_scan	L4
07 fused_causal_conv1d	L4
08 fft_conv	L4
09 lact_ttt_chunk	L5
10 aggregate_hidden	L3
11 linear_cross_entropy	L4

新增算子 12—21	Level
12 aggregate_hidden_grad	L3
13 mhc_post_grad	L3
14 dynamic_mx_quant	L4
15 q4_0_dequant_matmul	L3
16 embedding_hash_table_lookup_or_insert	L3
17 categorical_embedding_bag	L3
18 kv_cache_append	L3
19 scatter_block_update	L3
20 image_resize_normalize	L2
21 tensor_program_vm	L5

CANN AtomGit

Micro 集同步换新，覆盖本轮代表场景

规模保持 16 项：保留 12 项，退出 4 项，新增 4 项

新增 4 项

Image Resize Normalize — L2

图像缩放与归一化的一体化预处理

Dynamic MX Quant — L4

生成块级共享尺度，并完成 FP8 量化

Linear Cross Entropy — L4

联合处理线性投影与交叉熵损失

Kimi Delta Attention — L5

门控 Delta 注意力的状态更新与输出计算

退出 Micro 的 4 项

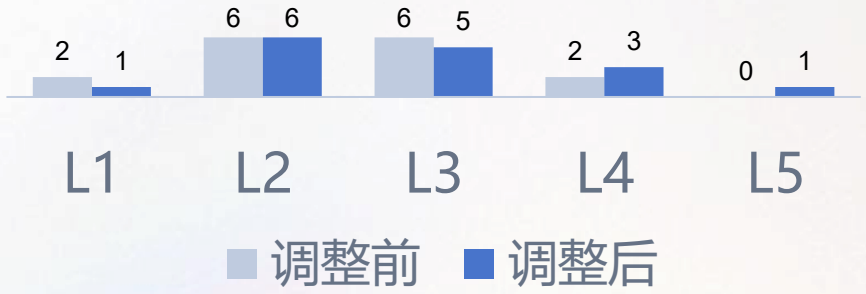
foreach_addcdiv_scalar

apply_rotary_pos_emb

dequant_swiglu_quant

mla_prolog

等级分布变化

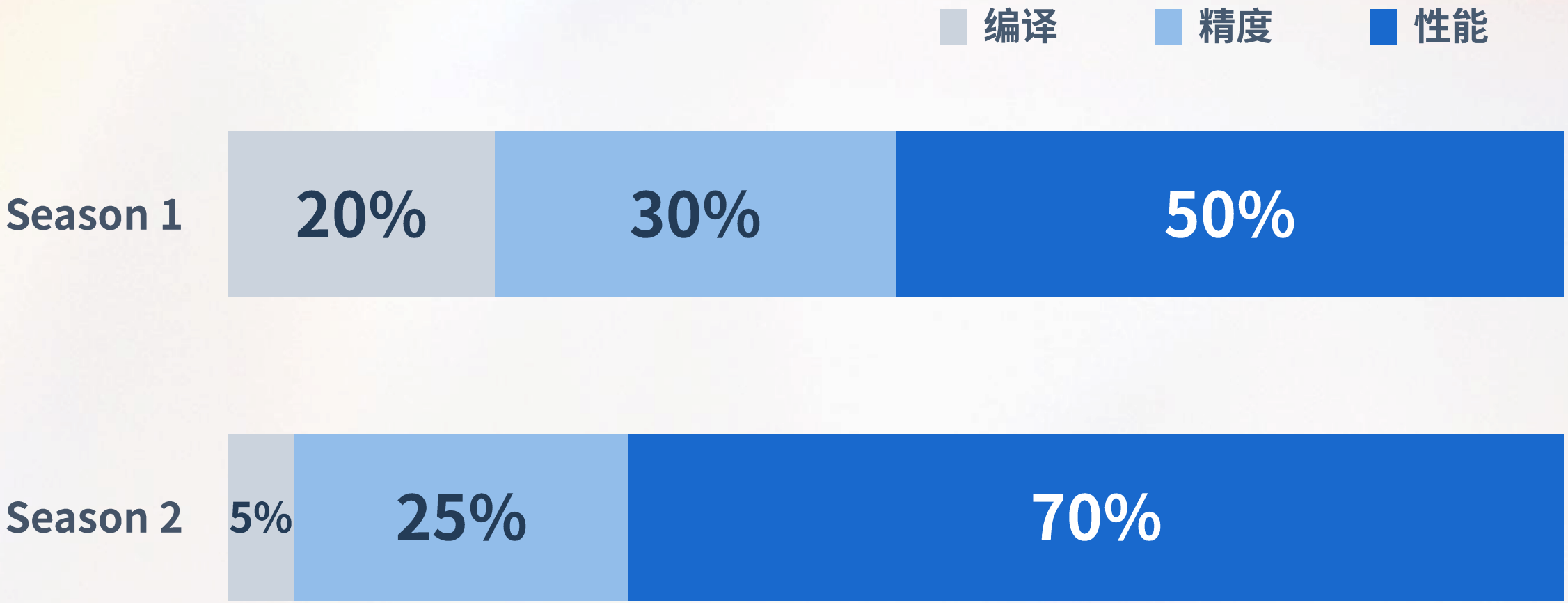


换入与换出不逐项配对。退出 Micro 的后 3 项仍保留在主集合，等级采用本轮规划口径

CANN AtomGit

评分机制进一步调整

智能体已具备确保算子可编译的能力，开始着重围绕性能提升进一步优化



CANN AtomGit

THANKS



高手论道算子生成　携手共探评测新季

CANN Bench Season 1 线下技术沙龙

2026.10.17　中国·南京

主办单位：CANN开源社区　协办单位：AtomGit

